Tel•Ran
Computing solutions LTD

תל•רן
פתרונות מיחשוב בעיימ

Methodological Guide

PROGRAMMING BASICS

The "Programming Basics" methodological guide is created by **Tel-Ran Educational Center**.

Compiled by: **Igor Korol**, **Alina Sokolova**, **Andrei Sher**.

Design and layout by **Andrei Sher**.

Special thanks to **Dr. Yuri Granovsky**, head of the methodological department of the educational center.

Rehovot, 2022

Content

Introduction

Before we start

Our brain is designed to remember important information and forget all the insignificant stuff. The only problem is that it considers learning to be a perfectly safe and therefore useless thing, and puts everything connected with potential danger to life (a tiger in the bushes!) on the most prominent shelf in the inner memory vault.

In order for your learning to be truly effective, you need to convince your brain that learning is actually something important and to make it work as intensely as possible!

We have prepared some recommendations that will help make your brain a partner and "explain" to it that information about programming is really important for you.

1. Get into the material, don't cram it.
The better you understand the explanation, the deeper you go into thinking and analyzing on your own — the more effectively your brain will work, and therefore the chances of memorizing the topic will increase.

2. Always do your homework.
In addition to practicing the material directly, problem solving helps to produce the hormone dopamine, a key element in the brain's "reward system.
It is also advisable to make notes in a notebook or notebook — it has been proven that handwritten explanations help you remember material better than typed ones.

3. Change places.
If you can't do your homework sitting at your computer desk, get up, stretch, walk around the apartment and choose a new place to work. This will help your body to shake up and your brain to look at the task from a different perspective.

4. Don't overload your brain after studying.
The process of learning and putting information into your long-term memory will take place partly during the class itself and partly afterwards. Therefore, in order not to interfere with the brain's ability to absorb the learning material, if possible, do not learn anything new (and especially not related to programming) after the study.

5. Discuss and talk through what you have learned.
Talking engages different parts of the brain, which means that the process of talking helps us in our goal! In a face-to-face class, discuss the topic during a break, ask questions, and look for answers on your own. In the case of remote classes, talk the main ideas out loud, or better yet, try explaining the topic to someone at home or a friend. Spell it out as you go along.

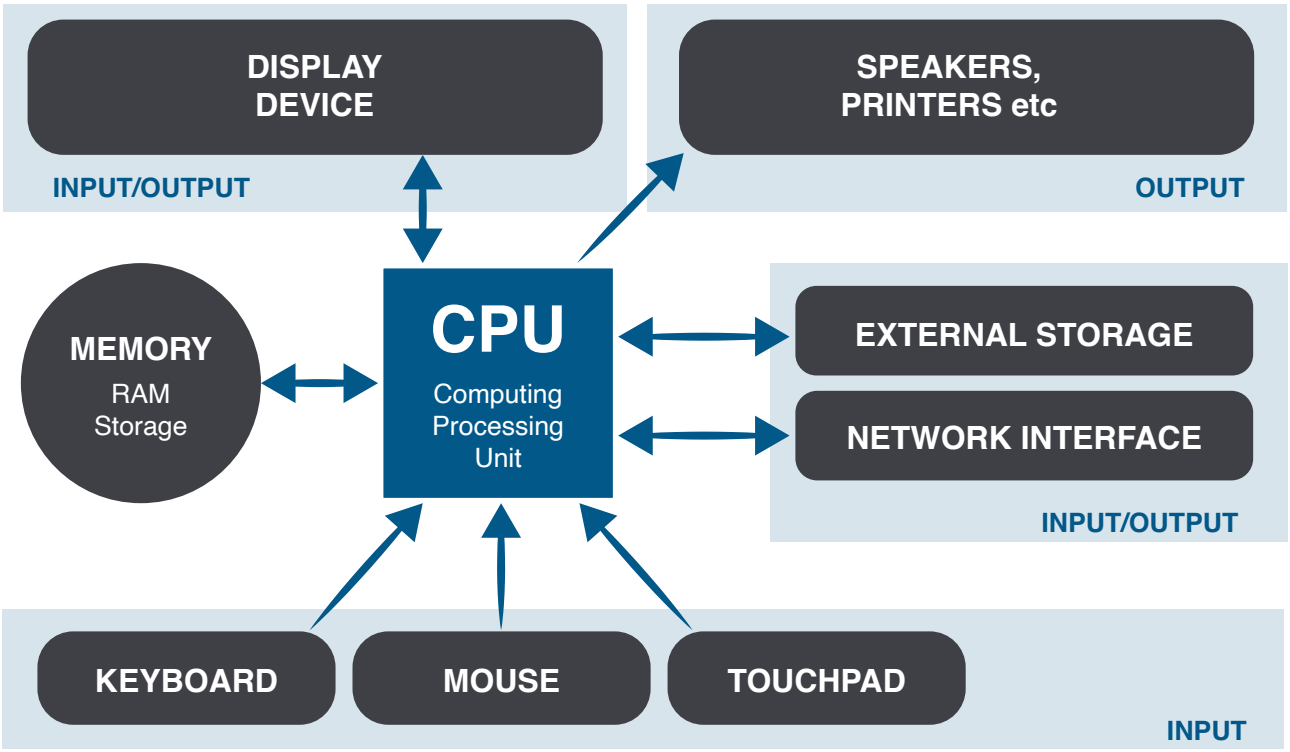
6. Listen to yourself.
Drink plenty of water: yes, it's a very cliché advice, but during hard work your brain really needs water. Take breaks: when the concentration is zero, and the material is impossible even to understand, let alone remember — do not blame yourself for laziness, but rather take a break. On average, after an hour of work a person needs at least 10-15 minutes break.

Have a fruitful studying!
Best wishes, your Tel-Ran Educational Center

Meet Mr. Sogerchik (aka Mr. Curly Bracket) — he is your virtual guide and helper on your way to learning programming. He can offer you additional tasks, tell you interesting facts or even joke around.

Personal Computer Devices

Schematic representation of input and output devices.



INPUT DEVICES: keyboard, mouse, touchpad, webcamera, microphone etc
OUTPUT DEVICES: display, printer, speakers etc

Numeral System — Notation

Set of techniques and rules for representing numbers using digital signs.

The number of possible characters in one position / digits

Computer binary presentation — yes/no

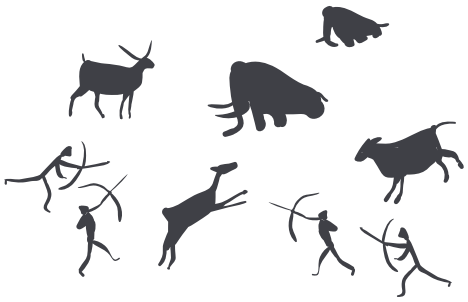
impulse / no impulse

Human presentation:

Decimal (Hindu-Arabic): zero - **0**, one - **1**, two - **2**, ... nine - **9**

Binary: 10111000111100

System	Base	Digits
Binary	2	0, 1
Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9



It all started with the unary number system. Since ancient times, people have been interested in numbers. People counted the number of days in a year, the number of constellations in the sky. To do this, they drew sticks, knots, etc.

Numeral System — Conversion

Exploring the most popular conversion systems — binary to decimal and vice versa.

Binary to Decimal Conversion

For binary number with **n** digits: $d_{n-1} \ldots d_3\ d_2\ d_1\ d_0$

$$dec = d_0 \times 2^{n-1} + d_1 \times 2^{n-2} + \ \ldots + d_{n-1} \times 2^0$$

Example

Find the decimal value of 111001_2:

binary number:	1	1	1	0	0	1
power of 2:	2^5	2^4	2^3	2^2	2^1	2^0

32 + 16 + 8 + 0 + 0 + 1

$$111001_2 = 1\times2^5+1\times2^4+1\times2^3+0\times2^2+0\times2^1+1\times2^0 = 57_{10}$$

Decimal to Binary Conversion

1. Divide the number by 2
2. Get the integer quotient for the next iteration
3. Get the remainder for the binary digit
4. Repeat the steps until the quotient is equal to 0

Example

Convert 13_{10} to binary:

Division by 2	Quotient	Remainder	Bit #
13/2	6	1	0
6/2	3	0	1
3/2	1	1	2
1/2	0	1	3

$$13_{10} = 1101_2$$

Dec	Bin
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010
11	1011
...	...
256	100000000

Think about the decimal number 4 with a binary representation of 100

Numeral Systems — Hexadecimal

The main advantage of the hexadecimal system is to reduce the digit capacity of numbers.

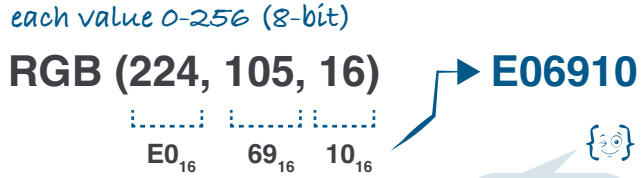
Hexadecimal Numbering System — What is it for?

The hexadecimal system is used quite extensively in modern information systems. For example, it is used to indicate colour scheme codes, it is used to record error codes, and it is also used for programming in low-level languages such as Assembler, and it is often used to provide data and addresses in low-digit computers.

Dec	Bin	Hex
0	00000000	0
1	00000001	1
2	00000010	2
3	00000011	3
4	00000100	4
5	00000101	5
6	00000110	6
7	00000111	7
8	00001000	8
9	00001001	9
10	00001010	a
11	00001011	b
12	00001100	c
13	00001101	d
14	00001110	e
15	00001111	f
16	00010000	10
17	00010001	11

Example — Hex Color Code

The clearest example for using the hexadecimal system is the Hex representation of the RGB color space. The code is written using a formula that turns each value into a unique 2-digit alphanumeric code.



So we see that any of the **16,777,216** different colors can be represented in hexadecimal code - six characters instead of at least nine. Hexadecimal colors are the visual language of the web. When you want a web page (or web application) to display a certain color, you give it a hexadecimal code.

Color	RGB	Hex
Black	(0, 0, 0)	#000000
Blue	(0, 0, 255)	#0000FF
Red	(255, 0, 0)	#FF0000
White	(255, 255, 255)	#FFFFFF

Units of Information

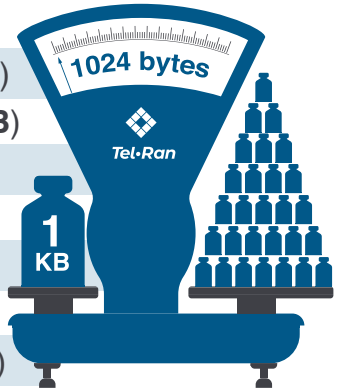
The capacity of some standard data storage system or communication channel.

Minimal information unit is one bit with two possible values: 0 or 1

The bit represents a logical state with one of two possible values.
These values are most commonly represented as either "1" or "0",
but other representations such as true/false, yes/no, +/–, or on/off are commonly used.

Byte is considered to be the smallest addressable unit of memory.

1024 bytes = 1 kilobyte (**KB**)
1024 kilobytes = 1 megabyte (**MB**)
1024 megabytes = 1 gigabyte (**GB**)
1024 gigabytes = 1 terabyte (**TB**)
1024 terabytes = 1 petabyte (**PB**)
1024 petabytes = 1 exabyte (**EB**)
1024 exabytes = 1 zettabyte (**ZB**)
1024 zettabytes = 1yottabyte (**YB**)



Non-programmer says that there are 1000 bytes in one Kilobyte and programmer says that there are 1024 meters in one kilometer

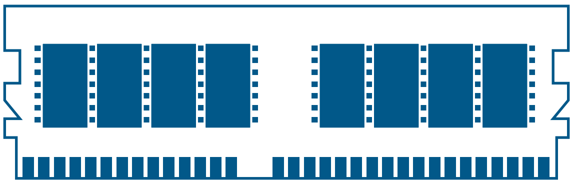
Random-Access Memory — RAM

It is a form of computer memory that can be read and changed in any order, usually used to store working data and machine code.

Address 0

10011011	10011011	10011011	00101110	00101110	00101111	00101101	11011011	01111000
10011011	10011011	10011011	00101110	00101110	00101111	00101101	11011011	01111000
10011011	10011011	10011011	00101110	00101110	00101111	00101101	11011011	01111000
10011011	10011011	10011011	00101110	00101110	00101111	00101101	11011011	01111000
10011011	10011011	10011011	00101110	00101110	00101111	00101101	11011011	01111000
10011011	10011011	10011011	00101110	00101110	00101111	00101101	11011011	01111000
10011011	10011011	10011011	00101110	00101110	00101111	00101101	11011011	01111000
10011011	10011011	10011011	00101110	00101110	00101111	00101101	11011011	01111000

Address 1073741824

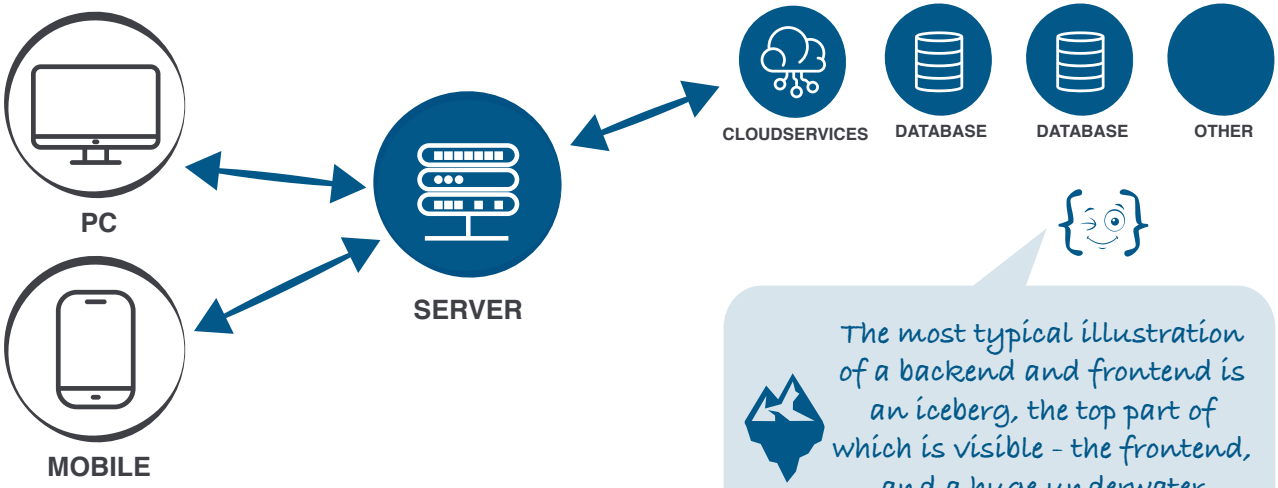


Schematic illustration of a modern RAM module

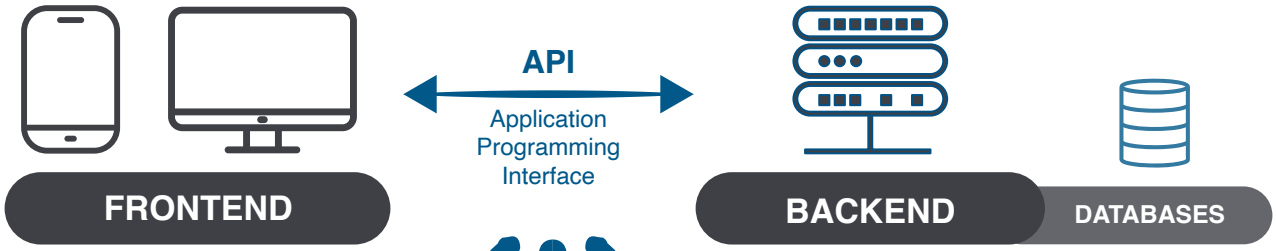
RAM is the main memory in a computer. It is much faster to read from and write to than other kinds of storage, such as a hard disk drive (HDD), solid-state drive (SSD) or optical drive. Random Access Memory is volatile.

Backend and Frontend

The backend and frontend can be a representation of the client-server model.
The client communicates directly with the user. The server (backend) is invisible to the user.



How are they interlinked and who does it?



Mobile
Swift
Kotlin
Java
etc

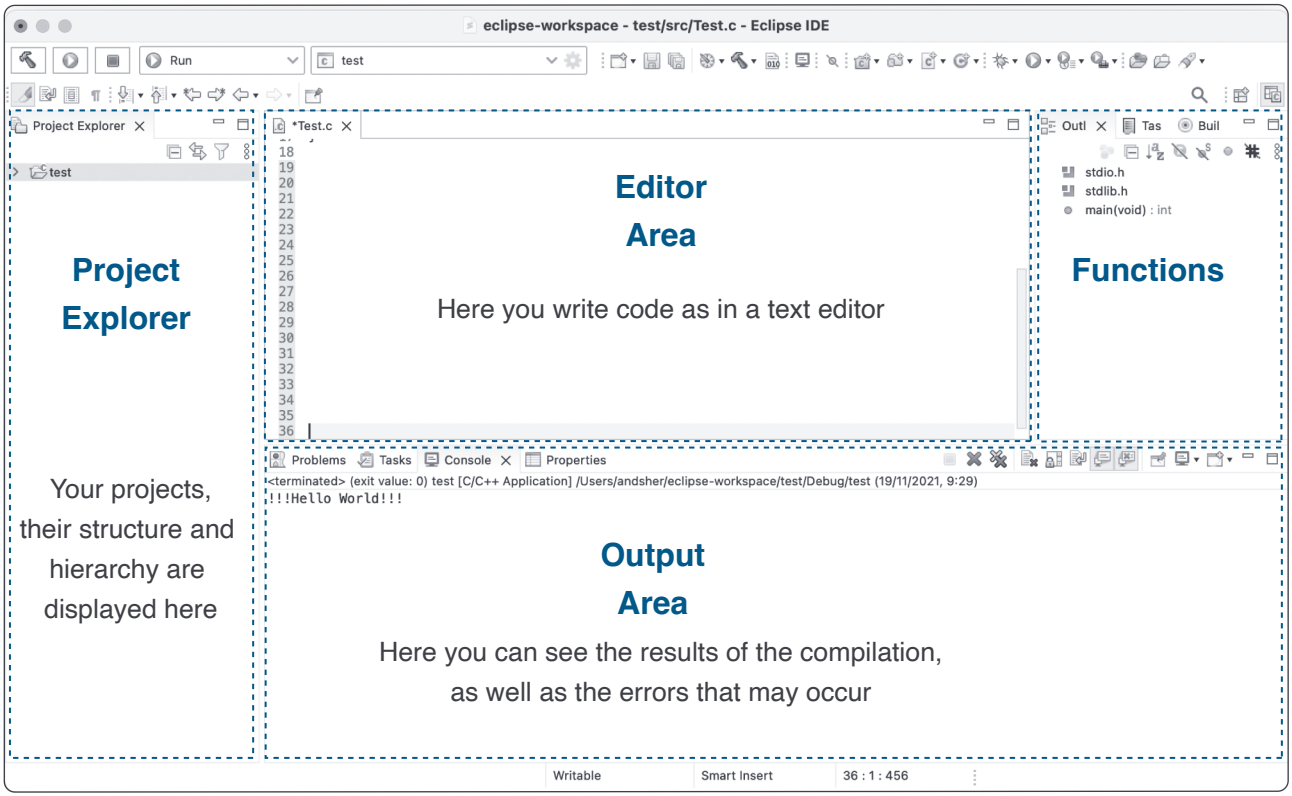
HTML, CSS
JavaScript
TypeScript
React
Angular
Vue
etc

Java
C#, Python
Node.js(JS)
Spring MVC
Express
Hibernate
etc

MySQL
MongoDB
Oracle
etc

Getting Started with the Eclipse IDE

An Integrated Development Environment (IDE) is a software application that provides programmers with complex software development capabilities.



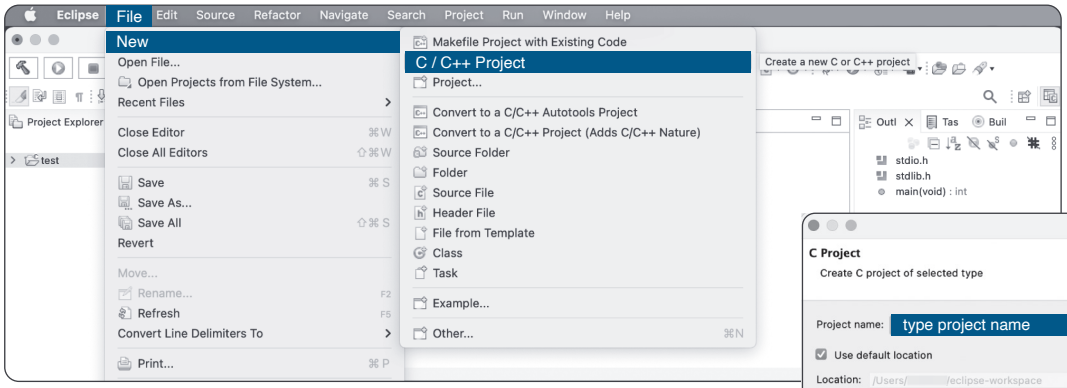
Any IDE is a more complex tool than a text editor. Despite the fact that text editors have a lot of useful features such as syntax highlighting, their only task is to provide work with code. It is important to add that you need at least a compiler and a debugger to be able to do full development. This is exactly what IDEs have.

There is no IDE that is "perfect". Each of them has its own advantages and disadvantages. It is fair to say that having experience with different IDEs is a huge plus for a programmer.

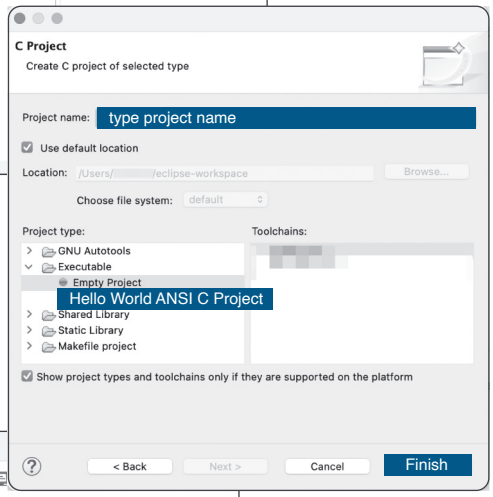
First Project with the Eclipse IDE ('C' programming language)

In just a few steps we are creating and compiling our first project — Hello World.

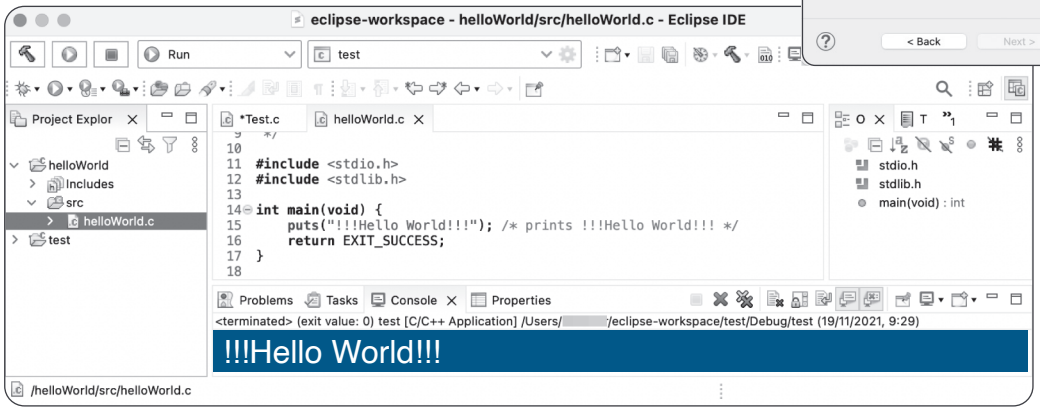
File -> New -> C Project



Type into 'Project name' helloWorld
Choose Hello world project
Press Finish



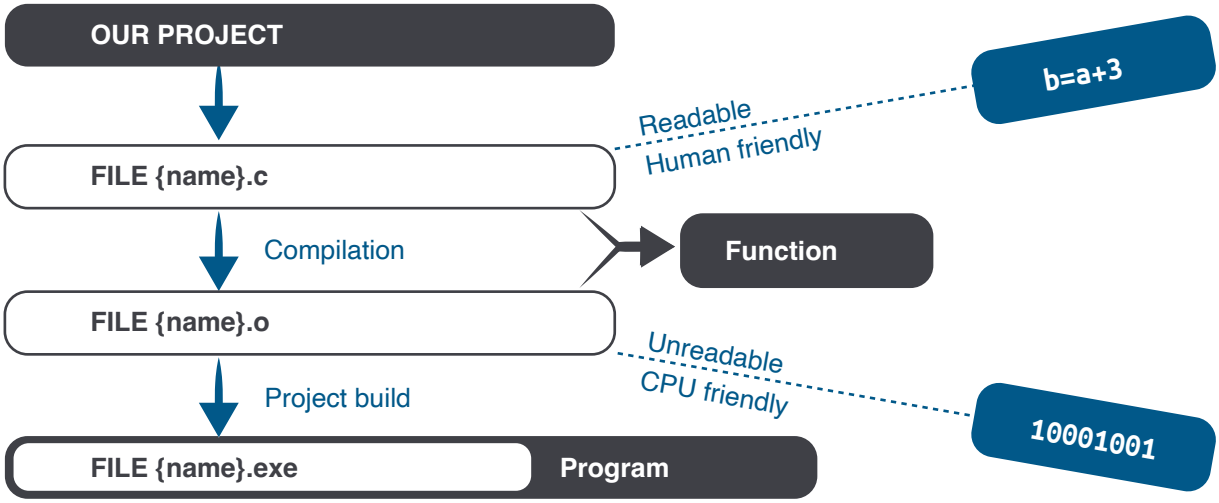
Project -> Build All
Run -> Run (mind the selection in Project Explorer)



Yay!
You have run your first program!

IDE — OS Interactions & New Project ('C' programming language)

Illustration of how our IDE interacts with the operating system — from the project to the running programme.



Let's create our first project and see how it works

File -> New -> C project
Select "empty project" ->
type **Test** "project name" ->
press "Finish"

File -> New -> Source File

Type **test.c** into
Source file -> press "Finish

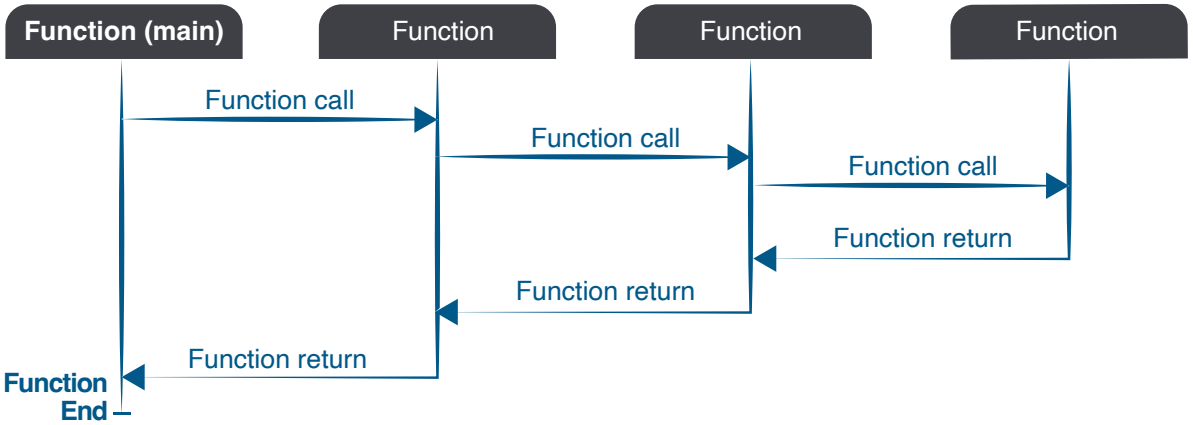
Type into editor area

Build All
Run

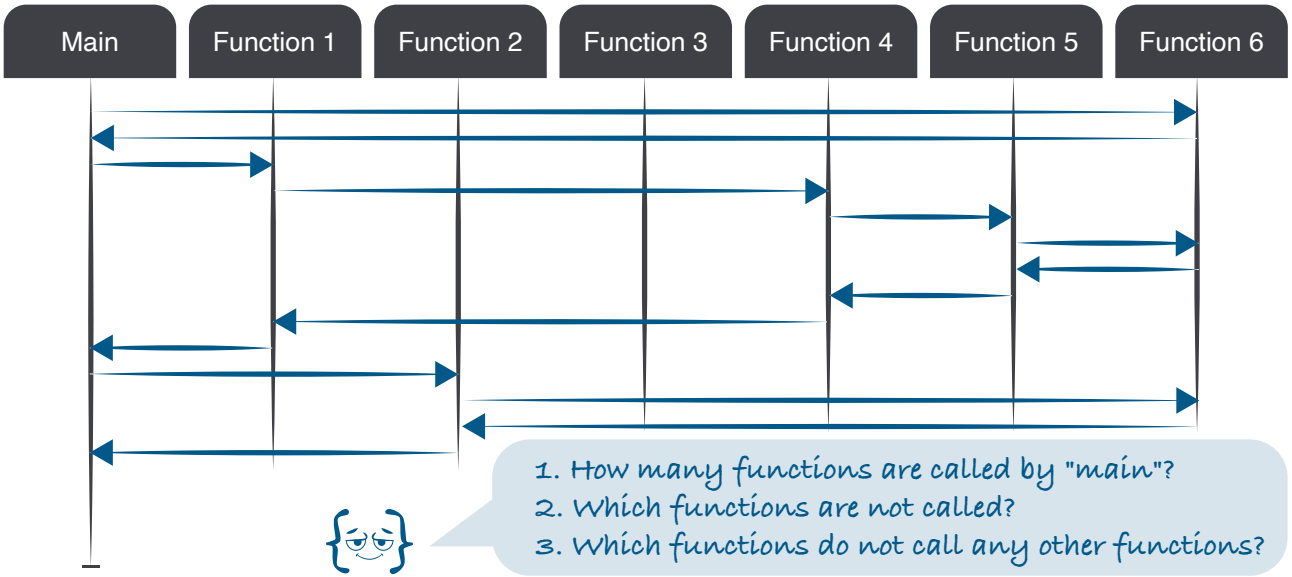
```
1  #include <stdio.h>
2
3  int multiply (int a, int b);
4
5  int main() {
6                  int res;
7                  res = multiply(2, 5);
8                  printf("Result is %d \n", res);
9                  return 0;
10 }
11
12 int multiply (int a, int b){
13                 int result;
14                 result = a*b;
15                 return result;
16 }
```

Function Call Sequence Illustration

Main function, function calls, return, sequence of execution.
Practical exercise on your own.

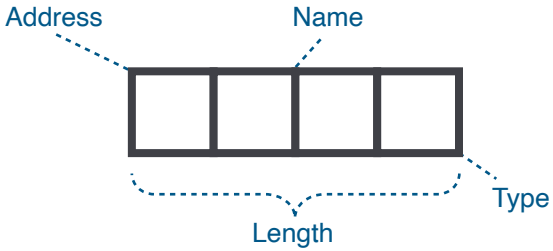


Practical exercise — Function Calls



Variables and Naming Rules ('C' programming language)

A variable is a name of the memory location. It is used to store data.
Its value can be changed, and it can be reused many times.



Basic variable types

`char`	Typically a single octet (one byte). Integer type.
`int`	The most natural size of integer for the machine.
`float`	A single-precision floating point value.
`double`	A double-precision floating point value.
`void`	Represents the absence of type.

Type
`int A;`
Name

Type defines the length (number of bytes) and operations that can be performed on the data contained in the memory.

Name defines a reference to a part of the memory which will be converted to an address.

Variable Declaration Example

```
int a;
float b;
char c;
```

or

```
int a=10,b=20;
float f=20.8;
char c='A';
```

w/o value | with value

Variable Naming Rules

starts with	followed by
`A - Z`	`A - Z`
`a - z`	`a - z`
_(underscore)	_(underscore)
	`0 - 9`

1. A variable name can have alphabets, digits, and underscore.
2. A variable name can start with the alphabet, and underscore only.
3. It can't start with a digit.
4. No whitespace is allowed within the variable name.
5. A variable name must not be any reserved word or keyword.

valid names	invalid names
`int a;`	`int 2;`
`int _ab;`	`int a b;`
`int a30;`	`int long;`

Which names are valid and which are not?

```
1  int 1ad          4  int Ad234_
2  int Ad-fr        5  int Df(32)
3  int ____         6  int a  b
```

Operators ('C' programming language)

An operator is simply a symbol that that tells the compiler to perform operations. There can be many types of operations like arithmetic, logical, bitwise, etc.

Arithmetic Operators (let's assume variable A holds 10 and variable B holds 20, then) *

+	Adds two operands.	A + B = 30
–	Subtracts second operand from the first.	A – B = -10
*	Multiplies both operands.	A * B = 200
/	Divides numerator by de-numerator.	B / A = 2
%	Modulus Operator and remainder of after an integer division.	B % A = 0
++	Increment operator increases the integer value by one.	A++ = 11
--	Decrement operator decreases the integer value by one.	A-- = 9

Relational Operators (let's assume variable A holds 10 and variable B holds 20, then)

==	Checks if the values of two operands are equal or not. If yes, then the condition becomes true.	(A == B) is not true
!=	Checks if the values of two operands are equal or not. If the values are not equal, then the condition becomes true.	(A != B) is true
>	Checks if the value of left operand is greater than the value of right operand. If yes, then the condition becomes true.	(A > B) is not true
<	Checks if the value of left operand is less than the value of right operand. If yes, then the condition becomes true.	(A < B) is true
>=	Checks if the value of left operand is greater than or equal to the value of right operand. If yes, then the condition becomes true.	(A >= B) is not true
<=	Checks if the value of left operand is less than or equal to the value of right operand. If yes, then the condition becomes true.	(A <= B) is true

Logical Operators (let's assume variable A holds 1 and variable B holds 0, then)

&&	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B) is false
ll	Called Logical OR Operator. If any of the two operands is non-zero, then the condition becomes true.	(A ll B) is true
!	Called Logical NOT Operator. It is used to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make it false	!(A && B) is true

* – We are not presenting examples of all existing operators, but only those we need in our basic programming course

Operators ('C' programming language)

An operator is simply a symbol that that tells the compiler to perform operations. There can be many types of operations like arithmetic, logical, bitwise, etc.

Assignment Operators*

=	Simple assignment operator. Assigns values from right side operands to left side operand	C = A + B will assign the value of A + B to C
+=	Add AND assignment operator. It adds the right operand to the left operand and assign the result to the left operand.	C += A is equivalent to C = C + A
-=	Subtract AND assignment operator. It subtracts the right operand from the left operand and assigns the result to the left operand.	C -= A is equivalent to C = C - A
*=	Multiply AND assignment operator. It multiplies the right operand with the left operand and assigns the result to the left operand.	C *= A is equivalent to C = C * A
/=	Divide AND assignment operator. It divides the left operand with the right operand and assigns the result to the left operand.	C /= A is equivalent to C = C / A
%=	Modulus AND assignment operator. It takes modulus using two operands and assigns the result to the left operand.	C %= A is equivalent to C = C % A

Precedence and Categories of Operators*

Category	Operator	Associativity
Postfix	() [] -> . ++ - -	Left to right
Unary	+ - ! ~ ++ - -	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Logical AND	&&	Left to right
Logical OR	ll	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=	Right to left

Differently from arithmetic, the "=" operator in programming is used to read from right to left, because it assigns the value of the right part to the left part.

```
int a = 21;
   int c;
   c = a;
   printf("Value of c = %d\n", c );
```

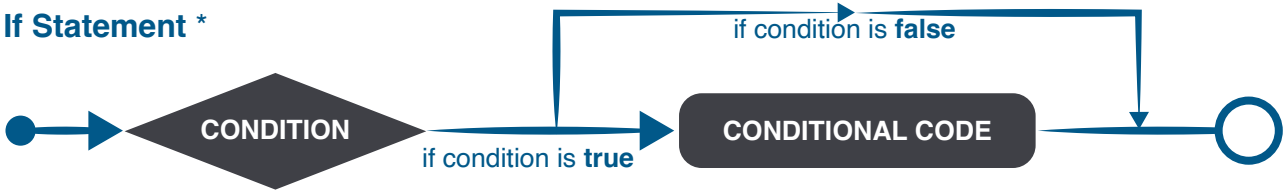
Value of c = 21

* – We are not presenting examples of all existing operators, but only those we need in our basic programming course

Decision Making Statements ('C' programming language)

The Condition Statement is used to perform the operations based on specific condition.

If Statement *

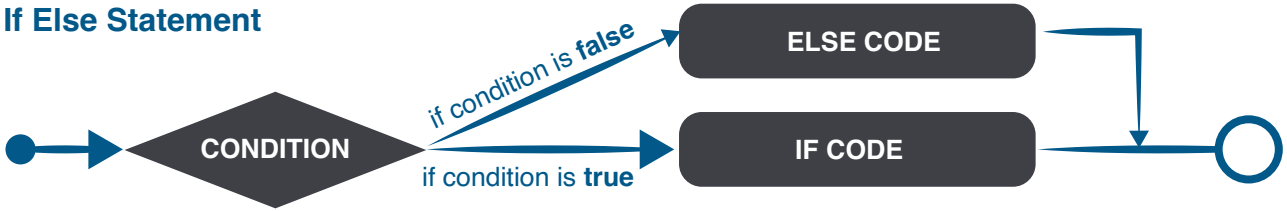


If the Boolean expression evaluates to true, then the block of code inside the 'if' statement will be executed. If the Boolean expression evaluates to false, then the first set of code after the end of the 'if' statement (after the closing curly brace) will be executed.

```
if (boolean_expression) {
   /* statement(s) will execute if the boolean expression is true */ }
```

```
1  int main() {
2        int a = 10;
3        if (a < 20) {
4            printf("a is less than 20\n");
5        }
6        printf("value of a is : %d\n", a);
7        return 0;
8  }
```

If Else Statement



If the Boolean expression evaluates to true, then the if block will be executed, otherwise, the else block will be executed.

```
if (boolean_expression) {/* statement(s)
    will execute if the boolean expression is
    true */}
        else {/* statement(s) will execute
        if the boolean expression is false */}
```

```
1  int main () {
2     int a = 100;
3     if( a < 20 ) {
4        printf("a is less than 20\n" );
5     } else {
6        printf("a is not less than 20\n" );
7     }
8     printf("value of a is : %d\n", a);
9     return 0;
10 }
```

* – We are not presenting examples of all existing statements, but only those we need in our basic programming course

Decision Making Statements ('C' programming language)

The Condition Statement is used to perform the operations based on specific condition.

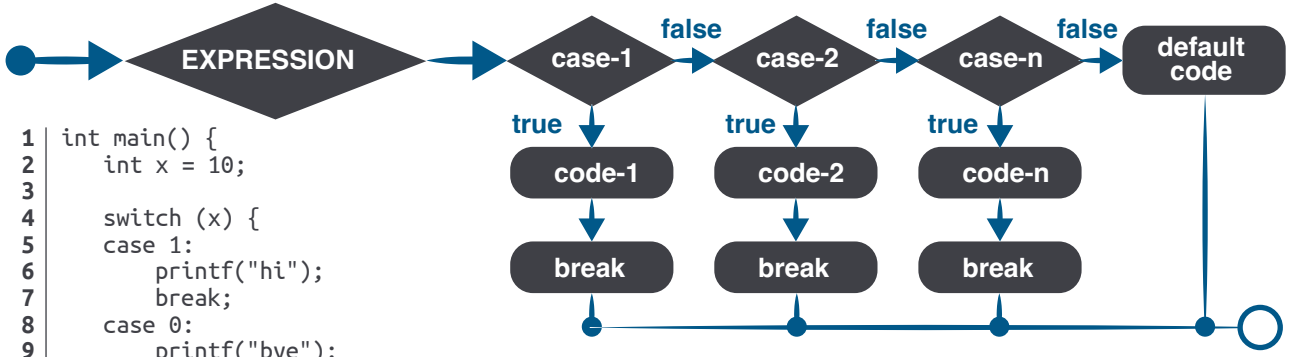
Conditional (Ternary) Operator — '?' and ':' *


```c
int main() {
   int a = 5, b;
   b = (a == 5) ? 3 : 2;
   printf("Value 'b' is: %d", b);
   return 0;
```

The behavior of the conditional operator is similar to the 'if-else' statement as 'if-else' statement is also a decision-making statement.

Switch Statement

A switch statement allows a variable to be tested for equality against a list of values. Each value is called a case, and the variable being switched on is checked for each switch case.




```c
int main() {
   int x = 10;

   switch (x) {
   case 1:
       printf("hi");
       break;
   case 0:
       printf("bye");
       break;
   default:
       printf(" Hello bye ");
 }
}
```

If we do not use the break statement, all statements after the matching label are also executed.

The default clause inside the switch statement is optional.

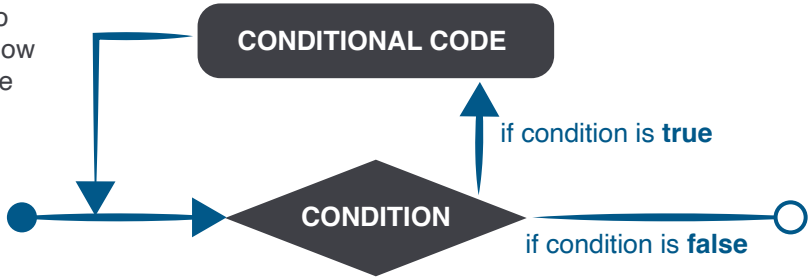
* – We are not presenting examples of all existing statements, but only those we need in our basic programming course

Loops ('C' programming language)

Looping is the repeated running of the same process until a certain condition is met.

Looping simplifies complex problems into simple ones. It allows us to change the flow of a program so that instead of writing the same code over and over again, we can repeat it a finite number of times.

There are three types of loops in C language: **do-while**, **while**, **for**.



Do-While Loop

```
1  int main() {
2      int i = 1;
3      do {
4          printf("%d \n", i);
5          i++;
6      } while (i <= 10);
7      return 0;
8  }
```

Using the **do-while** loop, we can repeat the execution of several parts of the statements.

The do-while loop is mainly used in the case where we need to execute the loop at least once.

While Loop

```
1  int main() {
2      int i = 1;
3      while (i <= 10) {
4          printf("%d \n", i);
5          i++;
6   }
7      return 0;
8  }
```

A **while** loop allows a part of the code to be executed multiple times depending upon a given condition.

The while loop is mostly used in the case where the number of iterations is not known in advance.

For Loop

```
1  int main() {
2      int i = 0;
3      for (i = 1; i <= 10; i++) {
4          printf("%d \n", i);
5   }
6      return 0;
7  }
8
```

The **for** loop in is used to **iterate*** the statements or a part of the program several times.

It is frequently used to traverse the data structures like the array and linked list.

* - Iteration is the repetition of a process to produce a sequence of results. Each repetition of the process is one iteration, and the result of each iteration is the starting point for the next iteration.

ASCII Table (American Standard Code for Information Interchange)

ASCII is a character encoding standard for electronic communication.

Encoding to represent decimal digits, Latin and national alphabets, punctuation marks, etc.
Each character has a numerical code ranging from 0 to 255 (one byte).

ASCII is often used in programming to define codes for characters pressed on a keyboard, or to encode/decode. It is not uncommon to see encoded characters in internet page addresses.

A character is a number.
Encoding is the correspondence between a character and a number.
One byte — one character corresponds to an ASCII code.
To use characters, you may use variables of the 'char' type.

```
char symbol = 'A';
```

```
1  int main() {
2     int count;
3     for (count = 0; count <= 127; count++) {
4         printf("ASCII value of %c is %d\n", count, count);
5   }
6     return 0;
7  }
```

Try running the program on the left, you already know the for loop and how it works. The result will clearly illustrate how ASCII works.

	1 49	2 50	3 51	4 52	5 53	6 54	7 55	8 56	9 57	0 48	- 189	= 187	Backspace 8
Tab 9	Q 81	W 87	E 69	R 82	T 84	Y 89	U 85	I 73	O 79	P 80	[219	] 221	
Caps Lock 20	A 65	S 83	D 68	F 70	G 71	H 72	J 74	K 75	L 76	: 186	' 222	Enter 13	
^ Shift 16	Z 90	X 88	C 67	V 86	B 66	N 78	M 77	< 188	> 190		^ Shift 16		
Ctrl 17		Alt 18	32						Alt 18	Ctrl 17	< 37	^ 38 v 40	> 39

Arrays (one-dimensional arrays in 'C' programming language)

Array is a kind of data structure that can store a fixed-size sequential collection of elements of the same type.

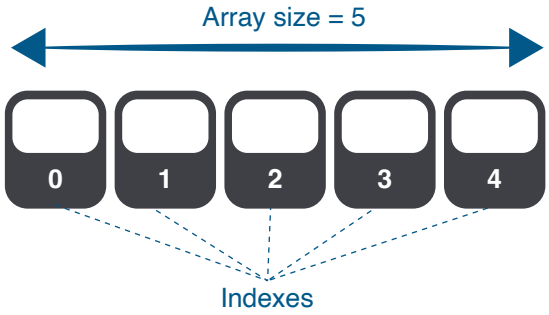
Array Declaration

If we want to store 100 integers, we can create an array for it.

```
int data[100];
```

dataType arrayName[arraySize];

Here we declared a data array of integer type. Its size is 100. This means that it can contain 100 integer values.



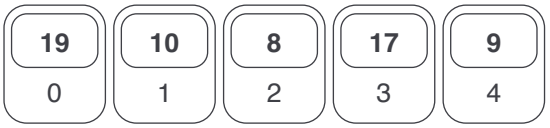
Array Initialisation

It is possible to initialize an array during declaration.

```
int ar[5] = {19, 10, 8, 17, 9};
```

or

```
int ar[] = {19, 10, 8, 17, 9};
```



Without specifying a size, the compiler knows that its size is 5 because we initialise it with 5 elements.

Accessing Array Elements

The elements of an array can be accessed by indexes. Example: you have declared an array **ar** as above. The first element is ar[0], the second element is ar[1], and so on.

```
1  int main() {
2          int ar[5] = {19, 10, 8, 17, 9};
3          printf("%d\n", ar[1]);
4          return 0;
5  }
```

Array Output

```
1  int main() {
2          int values[5] = { 19, 10, 8, 17, 9 };
3          for (int i = 0; i < 5; ++i) {
4              printf("%d\n", values[i]);
5          }
6          return 0;
7  }
```

Suppose you have declared an array of 10 elements: **int testArray[10];**
You can access the array elements from **testArray[0]** to **testArray[9]**.
What if you try to access **testArray[12]**?

Nested Loops ('C' programming language)

Nesting loops is a functionality that allows you to loop statements 'inside' another loop.

Syntax of Nested Loop

Outer_loop and **Inner_loop** are the valid loops that can be a 'for' loop, 'while' loop or 'do-while' loop.

```
Outer_loop {
      Inner_loop {/*inner loop statements.*/}
      /* outer loop statements*/
}
```

Nested do..while Loop Example

The nested do..while loop means any type of loop which is defined inside the 'do..while' loop.

```
1  int main() {
2        int i = 1;
3        int rows = 6;
4        int columns = 6;
5        do {
6            int j = 1;
7            do {
8                printf("*");
9                j++;
10           }
11           while (j <= columns);
12           printf("\n");
13           i++;
14       } while (i <= rows);
15       return 0;
16 }
```

Nested for Loop Example

The nested for loop means any type of loop which is defined inside the 'for' loop.

```
1  int main() {
2      int i, j;
3      int rows = 6;
4      int columns = 6;
5      for (i = 0; i <= columns-1; i++) {
6            for (j = 0; j <= rows-1; j++)
7  {
8                printf("*");
9            }
10           printf("\n");
11     }
12     return 0;
```

The output of the above examples of nested loops is a solid rectangle:

```
******
******
******
******
******
******
```

Try to output the following using any nested loops you already know:

```
******   ******   ******
*    *   **   *   **  **
*    *   * *  *   * ** *
*    *   *  * *   * ** *
*    *   *   **   **  **
******   ******   ******
```

Basic Array Operations (one-dimensional arrays in 'C' programming language)

Examples of array reversal, finding the minimum/maximum element of an array.

Array Reverse

Array reversal is important to understand. This can be helpful in job interviews. We can reverse array by using swapping technique.

Example of the original array of integers

Above array in reversed order

You will learn about alternative techniques and ways of reversing a data array in the professional part of the course.

```c
#include <stdio.h>

void reverseArray(int array[], int size);
void printArray(int arr[], int size);

int main() {
        int arr[] = { 10, 20, 30, 40, 50 };
        reverseArray(arr, 5);
        printArray(arr, 5);
        return 0;
}

void reverseArray(int array[], int size) {
        int i, j, tmp;
        for (i = 0, j = size - 1; i < j; i++, j--) {
            tmp = array[i];
            array[i] = array[j];
            array[j] = tmp;
        }
}

void printArray(int arr[], int size) {
        int i;
        for (i = 0; i < size; i++) {
            printf("[%d]", arr[i]);
        }
        printf("\n");
}
```

Minimal Array Element

```c
int findMin(int arr[], int size) {
        int min = arr[0];
        int i;
        for (i = 1; i < size; i++) {
            if (arr[i] < min) {
            min = arr[i];
             }
        }
        return min;
}
```

According to the given examples, write functions that return the maximum value of the array and the sum of the array elements:

```c
int findMax(int arr[], int size);
int arraySum(int arr[], int size);
```

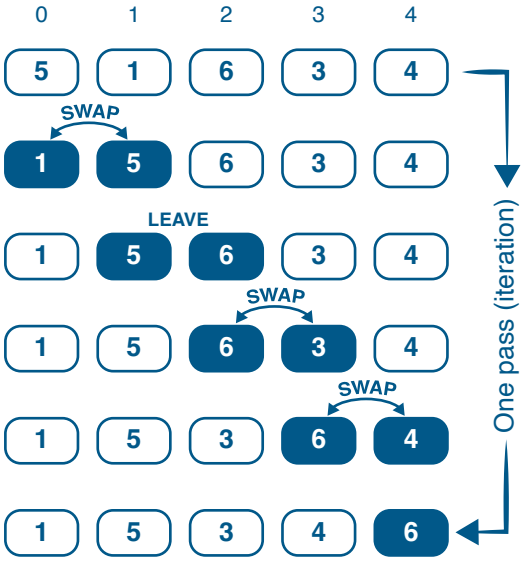
Array Sort (one-dimensional arrays in 'C' programming language)

Sorting an array means to arrange the elements in the array in a certain order.

Bubble Sort

Bubble sorting is the simplest sorting algorithm that works by repeatedly switching neighboring elements if they are in the wrong order.

Example of bubble sorting operation:



There are various types of sorting methods possible — Bubble sort, Selection sort, Quick sort, Merge sort, etc.

You can also discover Selection Sort as part of the Basic Programming course

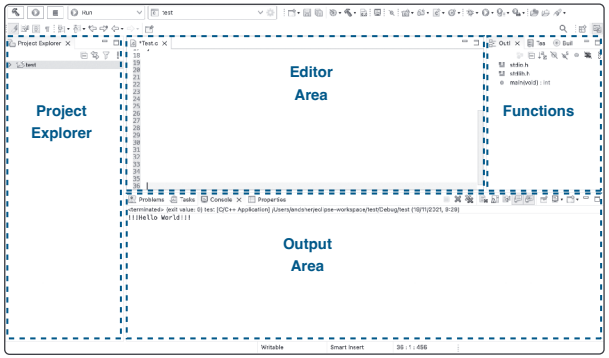
```
1  #include<stdio.h>
2
3  void printArr(int arr[], int size);
4  void bubbleSort(int arr[], int size);
5
6  int main() {
7       int arr[10] = { 10, 9, 8, 7, 6, 5, 4, 3, 2, 1 };
8       printArr(arr, 10);
9       bubbleSort(arr, 10);
10      printArr(arr, 10);
11      return 0;
12 }
13
14 void bubbleSort(int arr[], int size) {
15      int i, j;
16      int flag = 0;
17      for (i = 0; i < size - 1; i++) {
18          for (j = 0; j < size - 1 - i; j++) {
19              if (arr[j] > arr[j + 1]) {
20                  flag = 1;
21                  int tmp = arr[j];
22                  arr[j] = arr[j + 1];
23                  arr[j + 1] = tmp;
24              }
25          }
26          if (flag != 0) {
27              flag = 0;
28          } else {
29              break;
30          }
31      }
32 }
33
34 void printArr(int arr[], int size) {
35      int i;
36      for (i = 0; i < size; i++) {
37          printf("[%d]", arr[i]);
38      }
39      printf("\n");
40 }
```

Getting Started with Java

Developed in 1995, Java is one of the most popular programming languages used mainly for server side tasks. It remains in the top of the most demanded languages in the world.

What is Java?

Java is a C-like programming language and platform. Java is a high-level, robust, object-oriented, and secure programming language that is considered the most widely used in the world, in part due to its multiplatform approach.



IDEs and Interface

We continue to use Eclipse to work with Java. The interface has not changed and is well known to you.

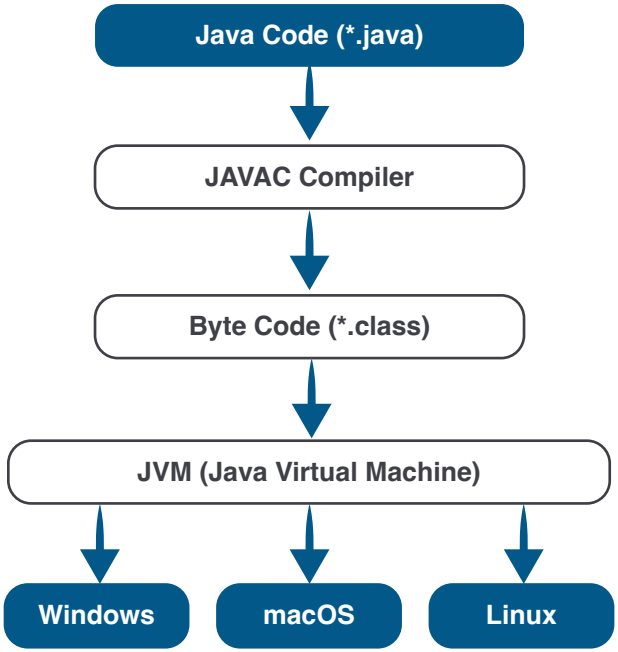
Java Platforms / Editions

1. Java SE (Java Standard Edition) — Java SE's API provides the core functionality of the Java programming language. It defines everything from the basic types and objects of the Java programming language to high-level classes.

2. Java EE (Java Enterprise Edition) — is built on top of the Java SE platform. The Java EE platform provides an API and runtime environment for developing and running large-scale, multi-tiered, scalable, reliable, and secure network applications.

3. Java ME (Java Micro Edition) — is a micro platform that is dedicated to mobile applications.

4. JavaFX —is used to develop rich internet applications written in Java FX Script. It uses a lightweight user interface API.



Getting Started with Java

A Java program is a sequence of Java instructions that are executed in a certain order. The Java instructions are executed in a certain order, a program has a start and an end.

The main() Method

A Java program begins by executing the main method of some class. You can choose the name of the class to execute, but not the name of the method. The method must always be called main.This is how the declaration of method main looks like.

```
1  public class AnyClass {
2
3      public static void main(String[] args) {
4
5          System.out.println("Hello Java");
6      }
7  }
```

This is the access modifier of the main method. It has to be public so that java runtime can execute this method.

Java main method doesn't return anything, that's why it's return type is void

This is the name of the java main method. It is fixed and must not be changed.

```
public static void main(String[] args) {  }
```

When java runtime starts, there is no object of the class present. That's why the main method has to be static so that JVM can load the class into memory and call the main method.

Java main method accepts a single argument of type String array. This is also called as java command line arguments.

Let's create our first project and see how it works

File -> New -> Java Project
Type project name as **first-project** -> Click Finish
Navigate to directory **src** inside your project
Right click on it -> New -> Class, Type Class name as **MyClass**
Click on checkbox below to include main() method -> Click Finish
In the body of main() method type System.out.println("Hello Java");
Click Save All -> Click Run

If it's not all clear, don't worry about it now. It will be explained later.

Just remember that the main() method declaration needs these keywords.

Java Data Types

Variables in Java must be of a certain data type. Data types are divided into two groups: Primitive Data Types and Non-Primitive (referential) Data Types.

Primitive Data Types

A primitive data type defines the size and type of variable values and has no additional methods. There are eight primitive data types in Java:

Data Type	Size	Description
byte	1 byte	Stores whole numbers from -128 to 127
short	2 bytes	Stores whole numbers from -32,768 to 32,767
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
boolean	1 bit	Stores true or false values
char	2 bytes	Stores a single character/letter or ASCII values

Non-Primitive Data Types (reference types)

Non-primitive data types are called reference types because they refer to objects. Examples of non-primitive types are **Strings, Arrays, Classes,** etc. Here we will have a look at the **String** type.

```
String myText = "Hello World";
```

String is a non-primitive type that stores its data as a table of characters (where each character is a primitive type - char). Strings are used for storing text.

String **Concatenation** in Java means, in general, merging, "gluing" two or more strings into one. Example: "basket" + "ball" is "basketball".

```
1  public class MyClass {
2     public static void main(String[] args) {
3
4          String s = "Hello";
5          String s1 = new String("Hello");
6          String s2 = "Hello";
7          System.out.println(s + " World");
8          // output: Hello World
9
10         int a = 5, b = 4;
11         System.out.println(a + b);
12         // output: 9
13         String s3 = "5", s4 = "4";
14         System.out.println(s3 + s4);
15         // output: 54
16    }
17 }
```

Java Methods and Parameters

A method is a block of code that is only executed when it is called.
We can pass data, called parameters, into a method.

Creating a Method

A method must be declared inside a class. It is defined by a method name followed by parentheses **()**. Java offers some predefined methods, such as **System.out.println()**, but we can also create our own methods to accomplish particular actions:

```
1  public class Main {
2    static void myMethod() {
3      // code to be executed
4    }
5  }
```

Calling a Method

To call a method in Java, write the method's name followed by two parentheses () and a semicolon;
myMethod();

```
static void myMethod()
```

static means that the method belongs to the Main class and not an object of the Main class.

myMethod() is the name of the method

void means that the method does not return any value

```
1  public class MyClass {
2
3      static void myMethod() {
4          System.out.println("Hello, Method");
5      }
6
7      public static void main(String[] args) {
8          myMethod();
9      }
10 }
```

A method can also be called multiple times:

```
public static void main(String[] args) {
        myMethod();
        myMethod();
        myMethod();
    }
```

Passing Data (parameters) into a Method

Information can be passed to methods as a parameter. Parameters act as variables inside the method. Parameters are specified after the method name, inside parentheses. We can add as many parameters as we want by simply separating them with a comma.

```
1  public class MyClass {
2     static void myMethod(String name, int age) {
3        System.out.println(name + " is " + age);
4   }
5
6     public static void main(String[] args) {
7        myMethod("Sarah", 10);
8        myMethod("Andrew", 15);
9        myMethod("Igor", 31);
10  }
11 }
```

parameters

arguments

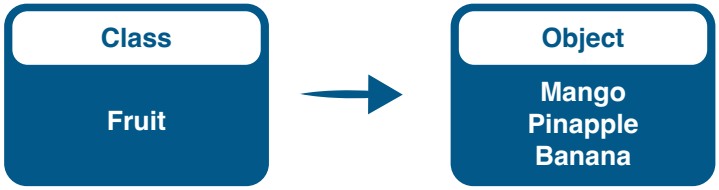
A method call must have as many arguments as parameters, and the arguments must be passed in the same order.

Java OOP — Object-Oriented Programming

OOP is a computer programming paradigm that arranges software development based on data, or objects, rather than functions and logic.

Classes and Objects

Classes and objects are the two main aspects of object-oriented programming. A class is a template for objects, and an object is an instance of a class.



Encapsulation

The point of encapsulation is to make sure that "sensitive" data is hidden from users. To achieve this you must: declare variables/attributes of a class as private, provide public get and set methods to access and update the value of a private variable.

```
1  public class Book {
2      private String author, title;
3      private int pages;
4
5      public void setAuthor(String author) {
6          if (author != null) this.author = author;
7      }
8
9      public String getAuthor() { return author; }
10
11     public void setPages(int pages) {
12         if (pages > 0) this.pages = pages;
13     }
14
15     public int getPages() { return pages; }
16
17     public void setTitle(String title) {
18         if (title != null) this.title = title;
19     }
20
21     public String getTitle() { return title; }
22
23     public void display() {
24         System.out.print("Author : " + author);
25         System.out.print("Title : " + title);
26         System.out.println("Pages : " + pages);
27     }
28 }
```

Getters and Setters

Get and Set

Private variables can only be accessed within one class (the external class cannot access them). However, it is possible to access them if we provide public get and set methods.

The get method returns the value of the variable and the set method sets the value.

Encapsulation Advantages

Better control over class attributes and methods.
Class attributes can be made **read-only** (get method) or **write-only** (set method).
Programmer can change one part of the code **without affecting** other parts.
Improved **data security**.

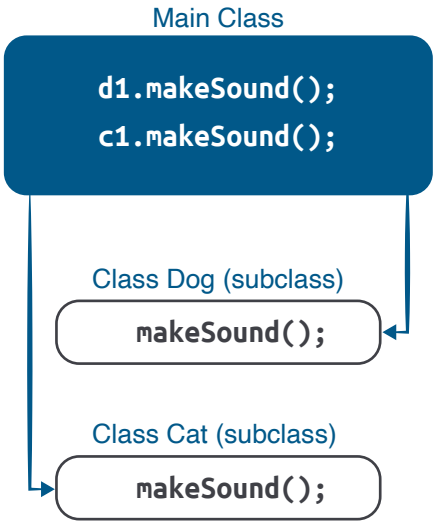
```
1  public class MainBook {
2      public static void main(String[] args) {
3
4          Book bk1 = new Book();
5          bk1.setAuthor("Yuval Noah Harari");
6          bk1.setTitle("Sapiens");
7          bk1.setPages(443);
8          bk1.display();
9      }
10 }
```

Java OOP — Polymorphism

Polymorphism means "many forms," and it appears when we have many classes linked to each other by inheritance.

Polymorphism

Polymorphism allows to simplify the use of created classes: the user of any of the two classes we created will know that to display a description of an object it is necessary to use the display() method, without even having to think about which object it is (i.e. which class it belongs to).



Why Polymorphism?

It is useful for code reuse and optimization - reusing attributes and methods of an existing class when creating a new class.

```java
public class Book {
    private String author, title;
    private int pages;

    public Book() { /* default constructor*/ }

    public Book(String author, String title, int pages) {
        this.setAuthor(author);
        this.setTitle(title);
        this.setPages(pages);
    }

    public void setAuthor(String author) {
        if (author != null) this.author = author;
    }

    public String getAuthor() { return author; }

    public void setPages(int pages) {
        if (pages > 0) this.pages = pages;
    }

    public int getPages() { return pages; }

    public void setTitle(String title) {
        if (title != null) this.title = title;
    }

    public String getTitle() { return title; }

    public void display() {
        System.out.print("Author : " + author);
        System.out.print("Title : " + title);
        System.out.println("Pages : " + pages);
    }
}
```

Constructors

Getters and Setters

```java
public class MainBook {
    public static void main(String[] args) {

        Book bk1 = new Book();
        bk1.setAuthor("Yuval Noah Harari");
        bk1.setTitle("Sapiens");
        bk1.setPages(443);
        bk1.display();

        Book bk2 = new Book("Goethe", "Faust", 512);
        bk2.display();
    }
}
```

Java OOP — Inheritance

Inheritance can be defined as the way in which one class acquires the properties (methods and fields) of another. It becomes manageable in a hierarchical order.

Inheritance

Inheritance in Java allows you to reuse the code of one class in another class, which means you can inherit a new class from an existing one. The main inherited class in Java is called the parent class, or superclass. The inherited class is called a child class, or subclass. A subclass inherits all the fields and properties of the superclass, and can also have its own fields and properties that are not present in the parent class.

```
public class A {
  ...
}
```

```
public class B extends A {
  ...
}
```

```
1  public class FictionBook extends Book {
2      private String genre;
3
4      public FictionBook() { }
5
6      public FictionBook(String author, String title, int pages, String genre) {
7          super(author, title, pages);
8          this.setGenre(genre);
9      }
10
11     public String getGenre() { return genre; }
12
13     public void setGenre(String genre) {
14         if (genre != null) this.genre = genre;
15     }
16
17     @Override
18     public void display() {
19         super.display();
20         System.out.println("Genre: " + genre);
21     }
22 }
```

from the previous page

```
1  public class MainBook {
2
3      public static void main(String[] args) {
4          Book bk2 = new Book("Goethe", "Faust", 512);
5          bk2.display();
6
7          FictionBook fbk1 = new FictionBook("Herbert", "Dune", 604, "Fantasy");
8          fbk1.display();
9      }
10 }
```

We've already been introduced to three of the four fundamental principles of OOP: Inheritance, Encapsulation, and Polymorphism. You can learn more about the abovementioned, as well as the fourth principle of OOP - Abstraction, in the professional part of our course. See you soon!

Congratulations

You did it!

You' ve completed the first important step toward your profession — completing the Basic Programming course. We want to wish you good luck in your further studies and give you 3 recommendations that helped many successful students to reach their goals.

1. You have been studying for several months, but you still have no results? In order not to lose your motivation to continue studying, you urgently need positive reinforcement. Positive reinforcement is not always praise, but also tasks already accomplished. Feeling accomplished, even if small, will bring you satisfaction and give you the understanding that everything is not in vain — because not long ago you knew nothing about loops, arrays, and variables!

2. What you focus your attention on becomes more significant in your life, and the quality of your life increases. The more time you devote to your future profession — the more projects you will finish! However, sitting and waiting for inspiration is a sure way to waste everything you've already started.

3. Perform your homework in Agile style: here and now, without thinking over the result for hours. In business and development, this principle is becoming the leading one. It is no longer necessary to spend years researching the market and doing one business session after another. You can take an idea, and immediately start implementing it — untested and obviously with mistakes. Same with homework: start doing it without thinking for hours about the task in theory. Through trial and error, you will get the right answer and achieve the best result!

We wish you all the best and look forward to seeing you at the next level.

Tel-Ran Educational Center

your new profession standard of living

Tel•Ran
Computing solutions LTD

Plaut, 10 (Science Park)
76706, Rehovot
Israel

Sderot ha-Histadrut, 25
3296016, Haifa
Israel

08-947-99-77
054-56-99-350

go@tel-ran.com

www.tel-ran.com